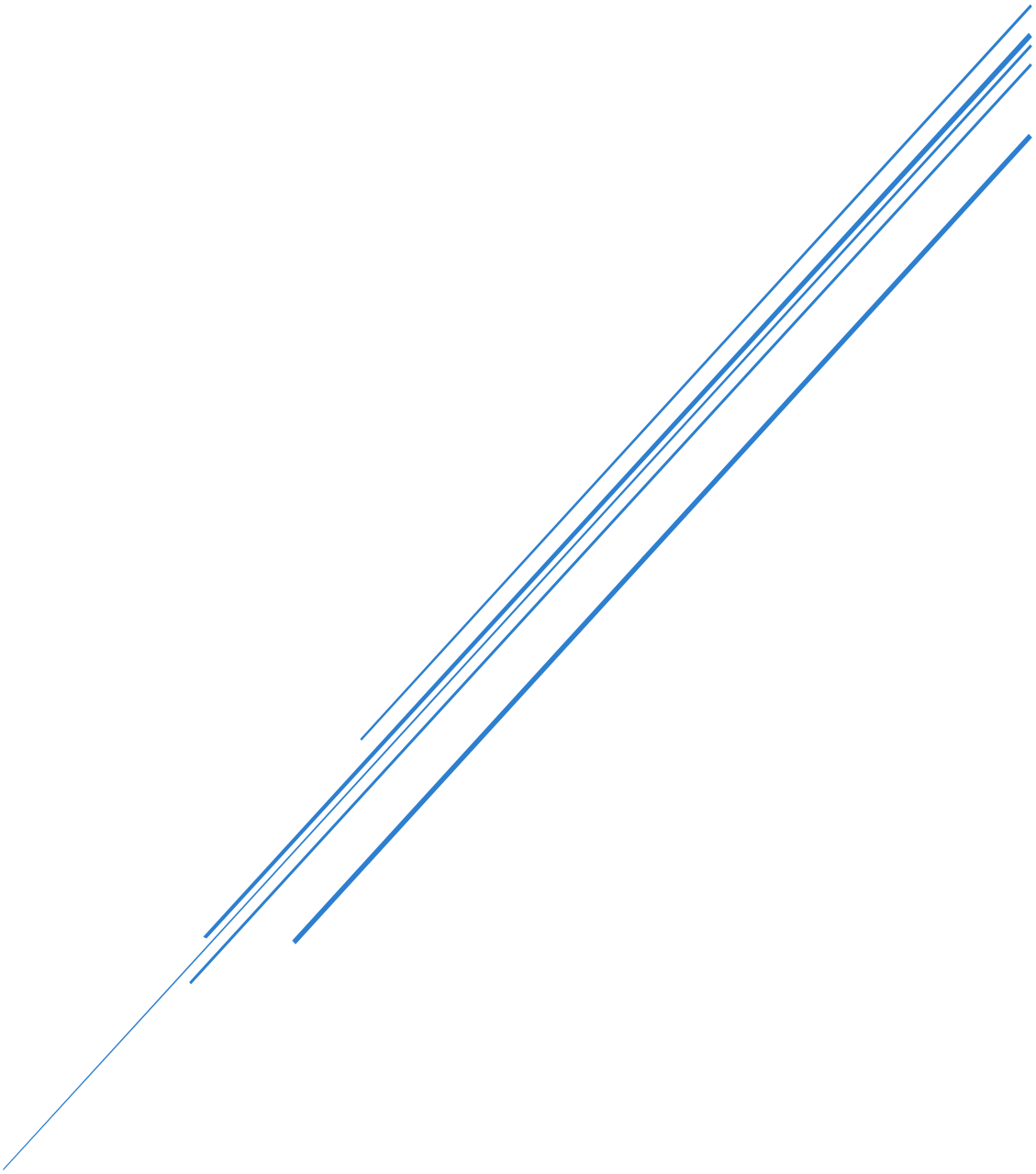
ARTEFACT 2 REPORT (A2)

Oliver Wuttke – WUTT0019



Flinders University, Tonsley

COMP3018: Probabilistic Machine Learning

Table of Contents

Table of Figures

Capstone and Team Integration Statement

The overarching system being built is a dashboard to be used by a quantitative trader to help them make decisions surrounding energy market investments. Each member will create their own Hidden Markov Model (HMM) to try and predict for some latent states relating to the energy sector. This report will cover a HMM that uses Geopolitical Risk (GPR) Index, XEJ (ASX 200 Energy) previous day close, and WTI crude oil close from the previous day to try and predict the regime of XEJ next day open (up, down, or flat). The same data contract from artefact 1 (A1) applies (Jan 1st, 2018 → Jan 1st, 2025) for training the HMMs. All these individual HMMs will (virtually, not actually implemented) make up individual modules that cover different latent states in the energy sector on a dashboard that a quantitative researcher or analyst could use to make informed decision based on the quantified uncertainty output of the HMMs to make investment decisions in the energy sector.

Introduction

Problem Statement and Objectives

Whilst A1 focused on a large Probabilistic Graphical Model (PGM), represented as a Directed Acyclic Graph (DAG). Artefact 2 (A2) focuses on breaking the PGM into smaller individual components where some nodes become latent states and others remain observed. This HMM (my individual module) has 3 discovered latent regimes (Baum-Welch / EM, unsupervised). At each day t the regime governs a multivariate Gaussian emission over the observed vector: GPR Index, XEJ close, WTI close. After training we interpret each regime by inspecting its learned emission means (which regime sits on positive XEJ moves, which on negative, which neutral), then map regimes to the up/flat/down output. The next-day-open direction is a derived prediction: take the inferred regime distribution at day t, push it one step through the transition matrix to get the regime distribution at $t + 1$, and read off the implied direction with its probability.

Sequential System Rationale

When trying to predict stock price movements (such as next day open/close movements) we are trying to model financial time series data. The issue with treating this data as independent and identically distributed (i.i.d.) is that financial time series isn't i.i.d., as it's often non-stationary and follows regime shifts/changes. If we were to assume our data is i.i.d. it would lead to a fixed distribution forever which could fail to capture precise uncertainty estimates during regime changes [1]. This means we need to think of a different approach that doesn't rely on the data being i.i.d. HMMs (and variants of HMMs) are an excellent tool for learning these regimes during training and then later inferring those regimes during inference, such as predicting next day open/close movements. This HMM approach allows for more robust modelling of our problem whilst providing quantified uncertainty with inference. The module we will be designing and implementing using HMMs is a textbook case for their use: we are trying to predict next day stock movements based on what we can observe today [2] [3].

Dataset and Modelling

Data Pre-processing

The pipeline assembles a daily feature frame on the XEJ trading calendar. Three raw inputs are joined: the monthly Geopolitical Risk GPR index, WTI oil close, and XEJ OHLC (Open, High, Low, Close) prices. All prices are $\log(x + 1)$ transformed (oil via a signed variant to tolerate the April-2020 negative print). Oil is left-joined onto XEJ trading days and forward-filled to bridge NYMEX/ASX holiday mismatches.

The central pre-processing decision is differencing. An initial attempt used observation levels, but these series are non-stationary and trend over multi-year horizons. The HMM consequently discovered slow "era" regimes, near-absorbing self-loops in the transition matrix that tracked epoch rather than behaviour, collapsing every state to the majority class. The final model instead uses three roughly stationary, zero-centred daily changes: month-over-month change in GPR_log , daily oil log-return, and daily XEJ close log-return.

Two further steps stabilise fitting. Winsorising clips each column to its [1, 99] percentile (bounds learned on train only) because the April-2020 oil shock otherwise produces an OIL_log outlier so extreme it captures an entire state by itself. Standardisation (train-only mean/std) then places all three features on a comparable scale for the Gaussian emissions. Splits are strictly chronological (train 2018–2022, test 2023, validation 2024) to prevent look-ahead leakage.

State Space

The model uses a 3-state latent space. States are unobserved and carry no prior meaning and are learned unsupervised via Baum-Welch (EM). Post-hoc analysis interprets them by their emission means and observation spread, yielding a calm / transitional / high-volatility ordering ranked by Oil_log dispersion. The latent states are mapped to (down, flat, up) directions after training, using only the train split's next-day open directions, and this mapping is then frozen.

Transition Probabilities

The transition probabilities are described by a 3×3 matrix A, where

$$A[i, j] = P(\text{state}_t = j \mid \text{state}_{t-1} = i),$$

is estimated by Baum-Welch.

Emission Probabilities

Each state emits observations under a multivariate Gaussian with a full covariance matrix, so emissions model both the mean change vector and the correlations between GPR, Oil, and XEJ Close within a regime.

Why Standard Approaches are Insufficient

A standard i.i.d. classifier treats each day's features independently and discards the order of observations. But financial volatility is famously clustered, calm and turbulent periods persist, so the conditional distribution of tomorrow depends on the regime the market is currently in, not just today's raw features [2]. The HMM addresses this by introducing a latent state with Markovian dynamics: the transition matrix encodes persistence, and filtered inference carries information forward without peeking ahead.

System Architecture and Implementation

Design

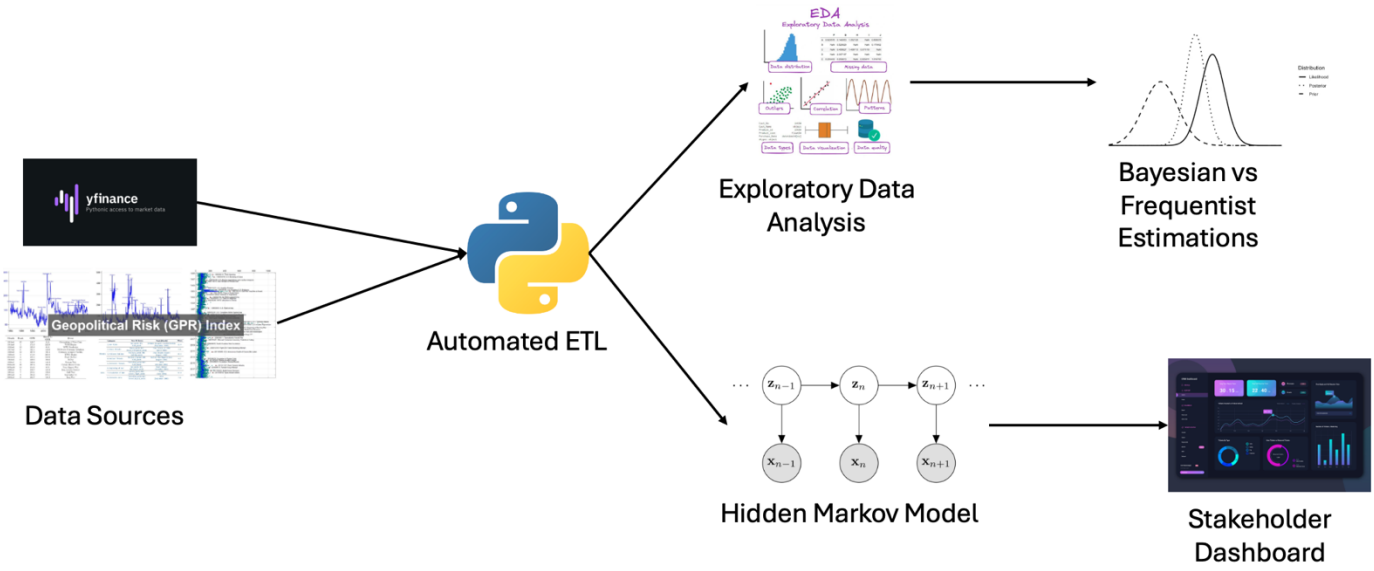


Figure 1: Overarching Probabilistic Inference Engine Architecture

Figure 1 shows the overarching system architecture from data source to stakeholder dashboards. Python scripts retrieve and automatically transform data ready for either the EDA + Bayesian vs Frequentist path or the HMM + stakeholder dashboard path. The EDA path is what A1 covered with automated exploratory data analysis and a Bayesian vs Frequentist estimation method comparison. The HMM path is what is covered in this report (A2) and can be seen in more detail in Figures 2 and 3.

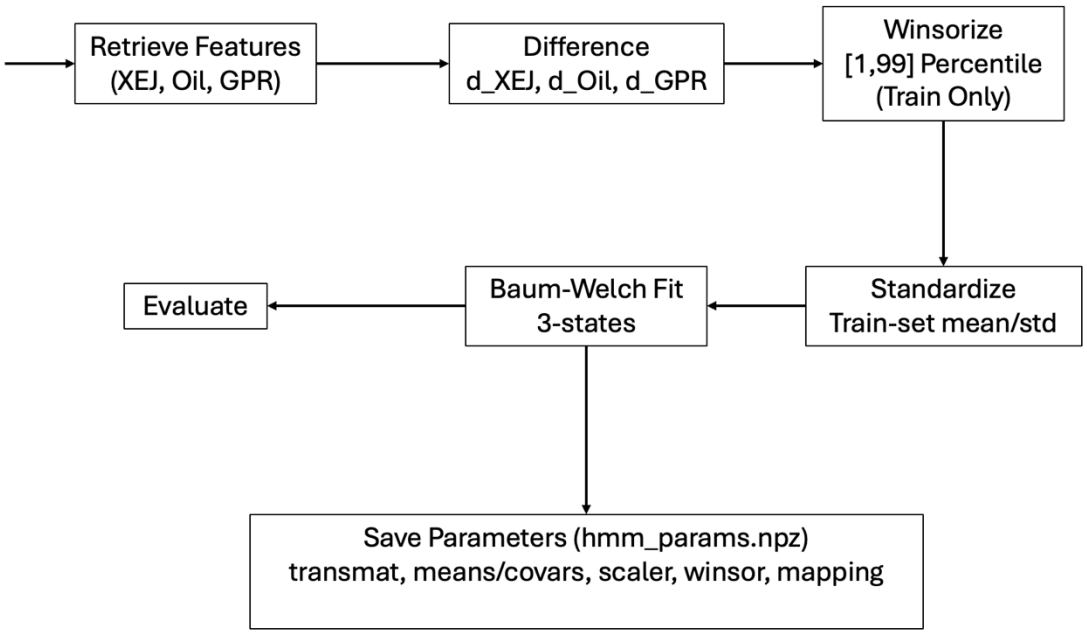


Figure 2: HMM Training Phase Diagram

Figure 2 outlines the initial (one-off) training phase for our sequential model; this is found in train_test_val.py. We start off by retrieving our features, then proceed compute the differences, winsorize to exclude the April-2020 oil price event, compute and apply a standard scaler, use Baum-Welch to fit to 3 states, evaluate our model against a variety of metrics, and finally save the HMM params to a .npz file so our inference model can use these without having redundantly do the training phase again.

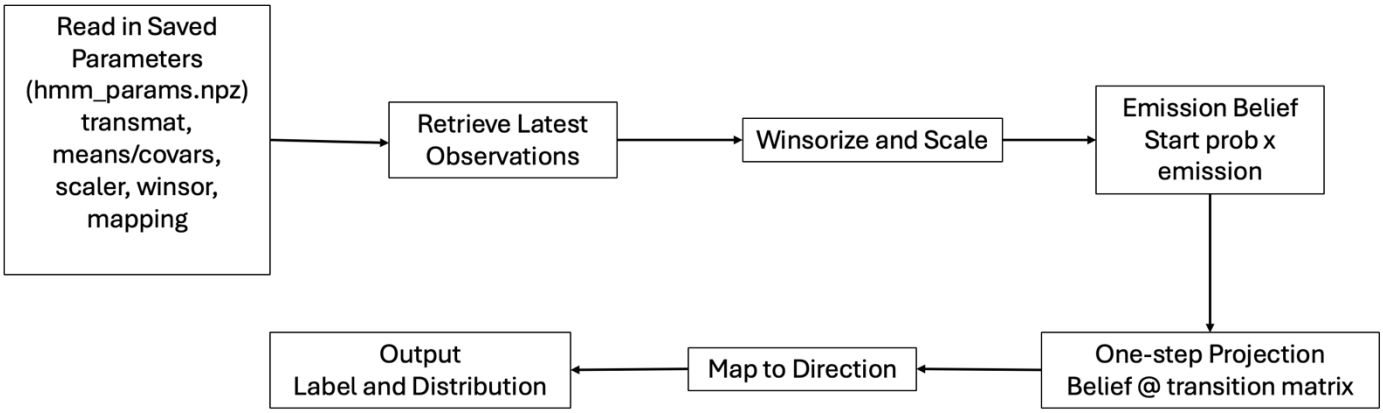


Figure 3: HMM Sequential Inference Diagram

Figure 3 shows the sequential inference process which runs once per prediction; found in probabilistic_model.py. We start off by reading in our params from training, then retrieve our latest observations, compute our emission belief, one-step projection which treats the latest day as a single-step belief rather than running the full forward pass, map to our discovered direction states, and finally output the label predictions alongside the probability distributions.

Implementation Details

The model is a 3-state Gaussian Hidden Markov Model fitted with hmmlearn's GaussianHMM. Parameters are the initial state distribution, the 3×3 transition matrix, and the per-state Gaussian emission means and covariances which are estimated on the chronological training split (2018–2022) via the Baum-Welch algorithm. Three inference problems arise in this pipeline, and a different algorithm is appropriate for each [3].

Learning the Parameters – Baum-Welch (Forward-Backward EM)

Because the latent volatility regimes are never observed, the parameters cannot be estimated by direct counting. The model.fit() function runs Baum-Welch: The E-step uses the Forward-Backward algorithm to compute smoothed state posteriors given the current parameters, and the M-step re-estimates the transition matrix and Gaussian emissions from those posteriors. The trade-off is that EM converges only to a local optimum, which is why the run is seeded.

Decoding – Viterbi

The first evaluation route calls model.predict(), which runs Viterbi to recover the single most probable state sequence over an entire split. Viterbi is the right algorithm when you want a coherent global path rather than independent per-step guesses, it maximises the joint probability of the whole sequence, so the decoded regimes respect the transition structure [3]. Viterbi uses the entire sequence past and future observations to decode any given day, so it is explicitly flagged in the code as an upper-bound descriptive fit, not a forecast.

Honest Forecasting – Filtered Forward-only One-step Projection
The predictive route (in both the training/evaluation phase and one-off prediction/inference phase) deliberately avoids Viterbi because a real next-day forecast cannot peek ahead. Instead, it uses the filtered posterior, the forward pass only. This is the honest metric because each prediction for day $t+1$ conditions only on information available up to day t.

Theoretical Computational Complexity Analysis

For our model parameters $N = 3$ states, $T =$ days, and $D = 3$ features, the following time and space complexity for our algorithms is given:

- Baum-Welch - Each EM iteration runs a full Forward-Backward pass costing $O(N^2T)$ time and $O(NT)$ space. This is repeated over several iterations until convergence giving $O(N^2T \times \text{iterations})$ time complexity. This is the most expensive phase, but it runs only once, so its cost does not affect prediction latency.
- Viterbi – Computing the most probable state sequence over a given split is an $O(N^2T)$ time complexity and $O(NT)$ space complexity operation. It is run only for evaluation and figure generation, not on the prediction path.
- Filtered one-step projection - A single prediction is a forward update plus one matrix-vector product through the transition matrix, this cost $O(N^2)$ time plus $O(D^2)$ (for the emission likelihood). This is the only algorithm on the live inference path, and it is effectively instant given our small N and D.

Every algorithm grows quadratically with the number of states but only linearly with sequence length T. At $N = 3$ and $D = 3$ the cost is negligible. A much richer regime space (50+ states) would be where this matters and the point at which exact inference gives way to approximate methods like particle filtering.

Conflicting Technical Requirements

The system trades statistical rigour against inference latency. Fitting the HMM via Baum-Welch over the full history is expensive, but a live next-day prediction must be cheap and history-free. The architecture resolves this by splitting the phases. Training serialises all frozen parameters like the transition matrix, emissions, scaler, winsor bounds, and state to direction mapping once to hmm_params.npz. Inference then loads this file and runs only a forward update plus a one-step projection on the latest observation no re-fitting, no historical re-computation. The costly exact estimation is paid once; the online path stays lightweight. At $N = 3$ the exact recursions are already fast enough that approximate methods would add error for no upside.

Evaluation and Analysis

Performance Metrics and Final Parameters

The final model is a 3-state full-covariance Gaussian HMM fitted on 1,264 training days, evaluated on 252 test (2023) and 253 validation (2024) days. Three state counts and both level and difference-based observations were trialled; the differenced 3-state configuration was retained because level observations produced near-absorbing "era" states with no behavioural meaning. The winsor bound, covariance floor and fixed seed were chosen to stop the April-2020 oil outlier from capturing a degenerate state and to make fitting reproducible.

Two evaluation routes are reported. The smoothed Viterbi decode is descriptive only, it uses the whole sequence to label each day and reaches 0.485/0.484/0.411 direction accuracy on train/test/val. The honest one-step-ahead forecast, which conditions only on information up to day t, reaches identical figures: 0.485/ 0.484/0.411. On every split this exactly equals the majority-class baseline (always predicting "up").

The probabilistic scores reinforce that this is not a useful forecaster. Log-loss is very high (14.2 nats on train/test, 16.3 on validation) and the multiclass Brier score is 1.03–1.18, both far worse than a three-way guess would achieve, the model is confidently wrong on the roughly 52% of days that are not "up". The drop from test (0.484) to validation (0.411) also shows the majority class itself is not stable across years.

The transition matrix (Figure 4) is the model's one clear success, the strong diagonals (0.97/0.96/0.97) show the HMM has learned genuinely persistent volatility regimes rather than noise. The regime characterisation supports the volatility-regime reading the three states differ sharply in d_Oil spread (0.031, 0.440, 0.026) and in mean GPR change. The states are real and interpretable they just don't encode next-day direction.

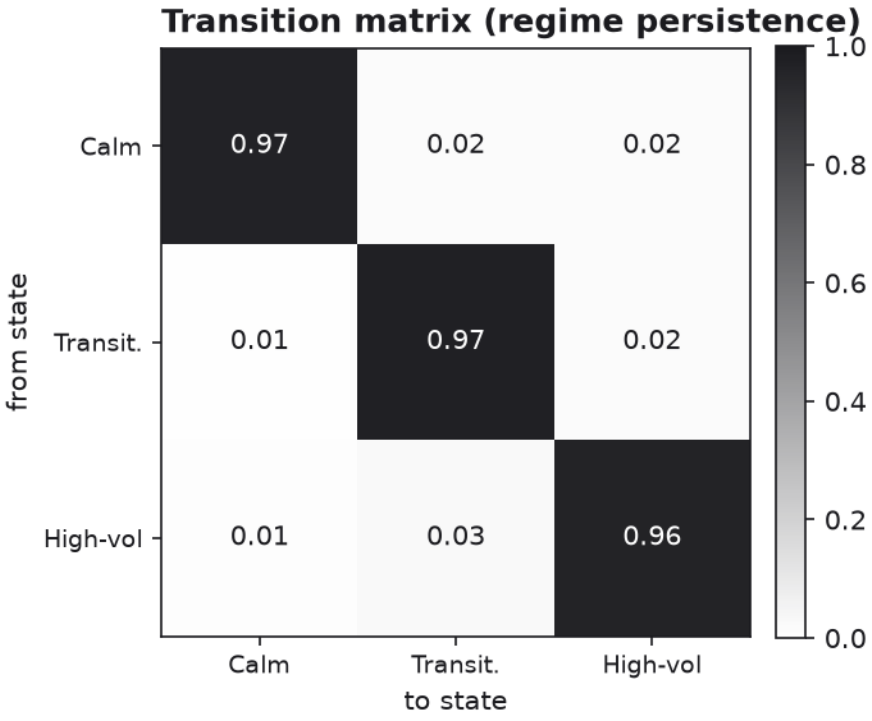


Figure 4: Transition Matrix

Overall Performance Evaluation

What the Markov model adds. The sequential structure recovers persistent, interpretable volatility regimes that an i.i.d. model cannot represent. The regime timeline (Figure 5) shows the high-volatility state aligning visibly with the 2020 oil shock, confirming that the latent states track volatility structure rather than noise. The value of the model is therefore in regime identification and honest uncertainty quantification, not point prediction. A deterministic classifier would emit a confident "up" with no warning, the HMM's near-flat posterior correctly signals that the directional signal is absent.

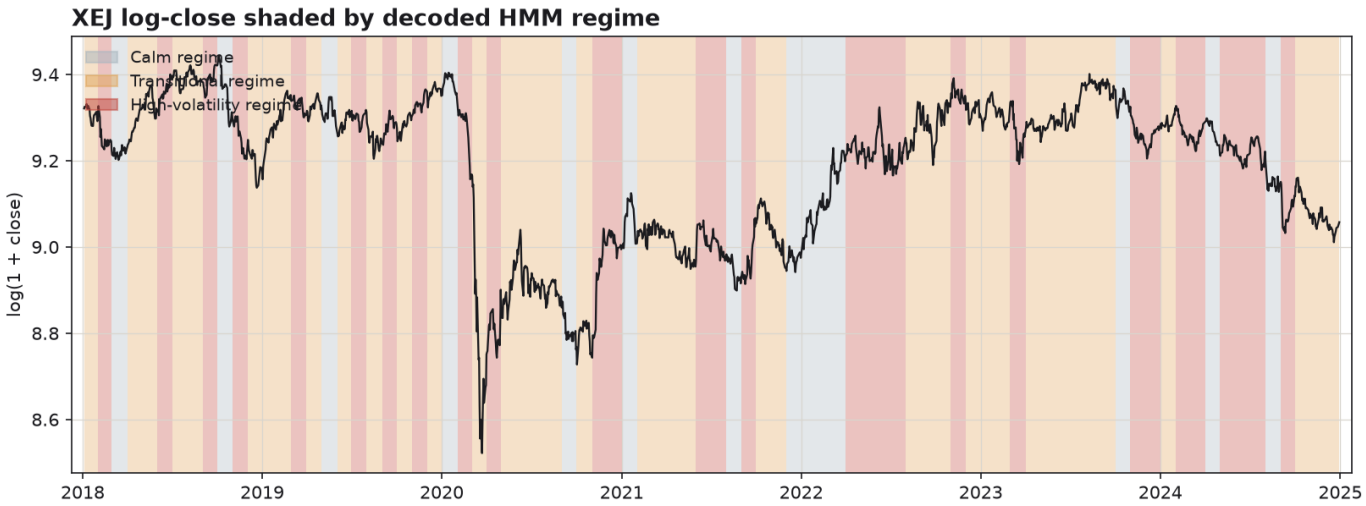


Figure 5: HMM Regime Timeline

The main limitation is the next-day direction mix is essentially identical across all three regimes (Figure 6: 0.44 down /0.07 flat /0.48 up in every state). This is the project's key finding is that the volatility regime is statistically independent of next-day open direction. The HMM cannot beat the majority class baseline because no relationship exists in these features.

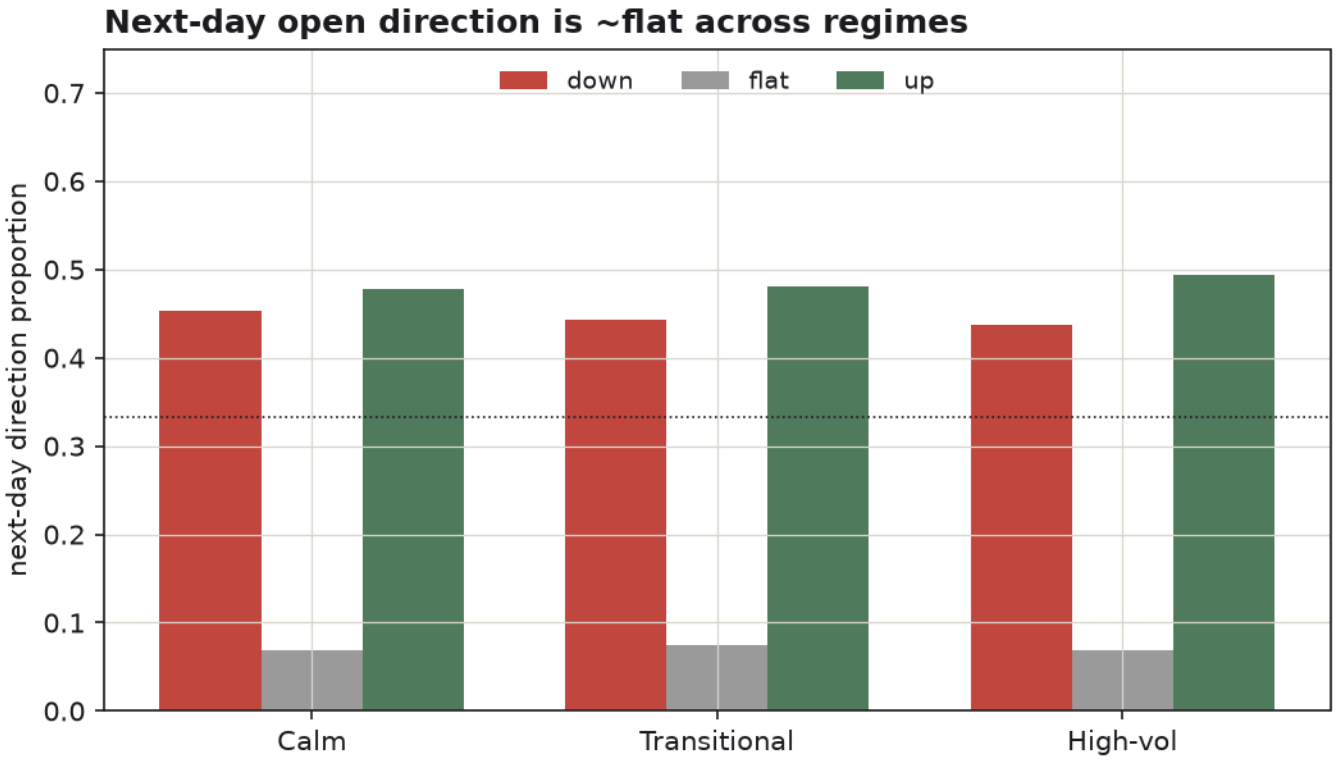


Figure 6: Next-day Open Direction Mix

Several modelling assumptions are stressed by this data. The Gaussian emission assumption is a poor fit for fat-tailed, skewed financial returns. The first-order Markov assumption compresses all history into the current state, discarding longer-range dependence [3]. Also, because every regime's plurality direction is "up", the argmax collapses to a single class and the model degenerates to the baseline.

For an energy-index trading or risk context, the significant failure mode is false confidence, a naive reading of the hard "up" label would suggest a usable directional edge that does not exist, and acting on it would lose money on the roughly 52% of non-"up" days. The system's redeeming property is that its quantified uncertainty makes this failure visible. An appropriate conclusion is not that the pipeline/model is broken, but that it correctly demonstrates the limits of next-day directional prediction from these inputs, and that the genuinely usable product is the volatility-regime signal, not the direction forecast.

Conclusion

This module implemented a 3-state Gaussian Hidden Markov Model to infer next-day XEJ open direction from geopolitical-risk, oil, and price features, using a two-phase architecture that separated offline Baum-Welch estimation from lightweight, history-free inference via frozen parameters and exact inference throughout.

The HMM recovers persistent, interpretable volatility regimes, strong transition diagonals, a high-volatility state aligning with the 2020 oil shock, but those regimes carry no next-day directional signal, leaving it unable to beat a majority-class baseline. Whilst the inference does collapse down to just predicting the majority state, the quantified uncertainty allows a user to make an informed decision based on this uncertainty instead of trusting the output of a deterministic classifier.

Future work follows from the limitations. Better emissions for fat-tailed returns, a higher-order or continuous latent space with approximate inference such as particle filtering for longer-range dependence, and most usefully reframing the target around volatility forecasting, turning the model's genuine strength into its objective.

References

[1] N. Nguyen, “Hidden Markov Model for Stock Trading,” *International Journal of Financial Studies,* vol. 6, no. 2, pp. 1-17, 2018.

[2] R. Hassan and B. Nath, “Stock Market Forecasting Using Hidden Markov Model: A New Approach,” in *5th International Conference on Intelligent Systems Design and Applications (ISDA’05)*, Melbourne, 2005.

[3] R. R. Lawrence, “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition,” *IEEE,* vol. 77, no. 2, pp. 257-286, 1989.

AI Usage Declaration

Permitted Uses of Artificial Intelligence: AI-Assisted Idea Generation & Structuring

AI was only used for idea generation and explanation of concepts, all work produced was written, coded, or produced by myself. All prompts and outputs were done using Anthropic's Claude Opus 4.8 Model https://claude.ai/.

Prompt	Response
Can you explain one-step projection in the context of a markov chain	For a Markov chain, one-step projection is matrix-vector multiplication. Your belief is a probability distribution over the states — a vector where each entry is the probability you're currently in that state, and the entries sum to 1. The transition matrix T holds the one-step transition probabilities. The standard convention is T[i][j] = P(go to state j | currently in state i), so each row sums to 1. To project one step forward, you multiply your belief (as a row vector) by T: $b' = b \cdot T$ Concretely, the probability of being in state j next step is: $b'[j] = \Sigma_i\, b[i] \cdot T[i][j]$ You're summing, over every state i you might currently be in, the probability you're there times the probability it sends you to j. That's just total probability / marginalizing over the current state. A small example with two states: b = [0.7, 0.3] T = [[0.9, 0.1], [0.4, 0.6]] b'[0] = 0.7·0.9 + 0.3·0.4 = 0.75 b'[1] = 0.7·0.1 + 0.3·0.6 = 0.25 b' = [0.75, 0.25]

	For k steps, just apply it k times: $b \cdot T^k$. And if you keep going, $b \cdot T^k$ converges to the stationary distribution π (where $\pi T = \pi$), assuming the chain is irreducible and aperiodic. Two things that trip people up: make sure you're consistent about row-vs-column convention (if T is column-stochastic instead, you'd do $T \cdot b$ with b as a column vector), and the belief stays normalized automatically here because each row of T sums to 1 — no renormalization needed, unlike the full Bayes filter where an observation update would require it.
Explain Baum-Welch and how it's used for computing posterior in HMMlearn	Baum-Welch is the expectation-maximization (EM) algorithm specialized to HMMs. It learns the model parameters when you only have observations and don't know the hidden states. The parameters it estimates For a discrete HMM: the initial distribution π, the transition matrix A (A[i][j] = P(state j next | state i now)), and the emission probabilities B (B[j][k] = P(observe symbol k | state j)). For a Gaussian HMM, B is replaced by the means and covariances of each state's emission distribution. The two pieces it relies on: forward and backward Baum-Welch is built on the forward-backward algorithm. The forward variable $\alpha_t(i)$ = P(o_1...o_t, hidden state at t = i) — the joint probability of the observations up to time t and being in state i at t. Computed left to right via the same one-step projection idea you just saw, but interleaved with emission reweighting: $\alpha_t(j) = [\Sigma_i \alpha_{t-1}(i) \cdot A[i][j]] \cdot B[j][o_t]$ The backward variable $\beta_t(i)$ = P(o_{t+1}...o_T | state at t = i) — the probability of the future observations given you're in state i at t. Computed right to left. The posteriors (this is what you're after) From α and β you get the two posterior quantities:

	$\gamma_t(i) = P(\text{state at } t = i \mid \text{all observations}) \propto \alpha_t(i) \cdot \beta_t(i)$, normalized over i. This is the posterior over a single state at one time. $\xi_t(i,j) = P(\text{state i at t, state j at t+1} \mid \text{observations}) \propto \alpha_t(i) \cdot A[i][j] \cdot B[j][o_{t+1}] \cdot \beta_{t+1}(j)$. This is the posterior over a transition. The EM loop E-step: run forward-backward with the current parameters to compute γ and ξ. M-step: re-estimate parameters as expected counts. New $\pi = \gamma_1$. New A[i][j] = (expected transitions i→j) / (expected times in i) = $\Sigma_t \xi_t(i,j) / \Sigma_t \gamma_t(i)$. New emissions are γ-weighted averages of the observations. Iterate until the log-likelihood stops improving. It's guaranteed to monotonically increase the likelihood, but only converges to a local optimum — initialization matters. In hmmlearn specifically hmmlearn (the Python library, e.g. GaussianHMM, MultinomialHMM/CategoricalHMM) implements Baum-Welch as the .fit(X, lengths) method. That's the full EM loop above — you give it observations, it learns π, A, and the emission parameters. lengths lets you pass multiple independent sequences concatenated together. For getting posteriors out of a fitted (or partially fitted) model: model.predict_proba(X) returns the $\gamma_t(i)$ — the posterior probability of each hidden state at each time step, shape (n_samples, n_components). This is the forward-backward posterior, the thing usually meant by "computing the posterior."

	model.score(X) returns the log-likelihood log P(observations | model), the forward pass's normalizer. model.predict(X) gives the single most likely path via Viterbi, which is different from the per-timestep posterior — Viterbi maximizes the joint over the whole sequence, whereas predict_proba marginalizes at each step. Don't conflate them: the argmax of predict_proba at each step need not equal the Viterbi path. One practical note: hmmlearn does everything in log-space internally (_do_forward_pass, etc.) to avoid the underflow you'd hit multiplying many small probabilities across a long sequence. If you ever implement this yourself, you'll need the same log-sum-exp trick or scaling factors.
Can you give a rough architectural outline for automated data ingestion into a HMM model for next day predictions of a stock	Here's a rough architecture. I'll lay it out as a pipeline, then flag the parts that matter most for an HMM specifically. 1. Ingestion layer A scheduled job (cron, Airflow, Prefect — whatever you're comfortable with) that pulls daily OHLCV data after market close from a price API. Land the raw response somewhere immutable first (a raw table or object store), then transform. Keeping raw separate from processed saves you when a vendor revises data or you find a bug in your feature code and need to recompute without re-pulling. 2. Storage Two logical zones: raw (append-only, exactly as fetched) and processed/feature store (cleaned, with engineered features, keyed by ticker + date). For a single stock a Parquet file or SQLite is plenty; don't over-build this. 3. Feature engineering — this is where HMMs live or die

	You almost never feed raw price into an HMM. Price is non-stationary (it trends, the mean drifts), and HMMs assume the emission distribution per state is stable. Use stationary, returns-based features instead: log returns: log(close_t / close_{t-1}) maybe realized volatility (rolling std of returns), volume change, a momentum measure Standardize/scale them. The classic setup is a GaussianHMM over a small vector of daily returns + volatility, where hidden states end up corresponding to market "regimes" (e.g. low-vol bull, high-vol bear). Critical: all features must be causal — computed only from data available up to and including day t. A rolling window that peeks at future bars is the single most common way people accidentally get amazing backtests that vanish live. 4. Training / refit Periodically refit (.fit()) on a trailing window of history — a rolling or expanding window. Decide your cadence: weekly/monthly refit is common; refitting daily makes the learned states unstable and hard to interpret. Persist the fitted model (pickle/joblib) with a version + training-date stamp so predictions are reproducible and auditable. Because Baum-Welch only finds a local optimum, refit several times with different seeds and keep the best by log-likelihood, and watch for state-label switching across refits (state 0 today may not be state 0 tomorrow). 5. Inference — "next day" This is the part to be careful about, because an HMM doesn't natively predict a price. The honest pipeline is:

	Run forward filtering on data through today to get the current belief over states: that's the filtered γ at the last step (or a forward-only predict_proba-style pass — note predict_proba uses full forward-backward, which uses future data; for a true next-day forecast you want forward-only filtering so you're not leaking). Project one step: belief' = belief · A (exactly the one-step projection from earlier). Map that projected state distribution to an expected observation: E[return_{t+1}] = Σ_j belief'[j] · (mean return of state j). That gives you an expected next-day return / direction, and you can also get a variance for uncertainty. So the output is a distribution over tomorrow's return, not a point price. Treat it as such. 6. Serving / output Write the prediction (date, expected return, state probabilities, confidence) to a table or dashboard. Log everything so you can compare predicted vs realized later. 7. Monitoring Track rolling prediction error, log-likelihood of new data under the current model (a drop signals regime change / model staleness → trigger refit), and feature drift. A diagram might make the data flow and the train/infer split clearer — want me to sketch one? And two honest caveats worth stating plainly: daily next-day stock prediction from an HMM on returns is, in practice, very weak — markets are close to a random walk at that horizon, and HMM regime models are usually more useful for risk/volatility regime detection than for directional return forecasting. Also, all of the above is about building a modeling pipeline, not financial advice; I'm not a

	licensed advisor, and a working pipeline producing a number is not evidence the number is profitable.